

AeroCode

Relatório de Qualidade — Métricas de Desempenho

Sistema de Gestão da Produção de Aeronaves

Disciplina: Desenvolvimento de Software para Web

Professor: Eng. Dr. Gerson Penha

Atividade de Avaliação Individual 3

Data: Junho de 2026

1. Introdução

Este relatório apresenta as medições de desempenho do sistema AeroCode, uma aplicação web desenvolvida para o gerenciamento da produção de aeronaves. Por tratar-se de um sistema crítico — cujas falhas podem impactar processos de fabricação aeronáutica — a verificação da qualidade por meio de métricas objetivas é obrigatória para garantir confiabilidade e desempenho adequados.

As métricas avaliadas são: **latência**, **tempo de processamento** e **tempo de resposta**, medidas sob três cenários de carga: 1, 5 e 10 usuários simultâneos. Todos os valores são expressos em milissegundos (ms).

2. Definição das Métricas

2.1 Latência

Atraso de rede entre o cliente e o servidor — o tempo que um dado leva para percorrer o caminho entre o navegador do usuário e o servidor da aplicação, excluindo o tempo de processamento. Calculada como: $\text{Latência} = \text{Tempo de Resposta} - \text{Tempo de Processamento}$.

2.2 Tempo de Processamento

Período em que o servidor trabalha para atender à requisição: execução da lógica de negócio, consultas ao banco de dados e preparação da resposta. Medido internamente pelo middleware do servidor e transmitido ao cliente via cabeçalho HTTP `X-Processing-Time-Ms`.

2.3 Tempo de Resposta

Tempo total percebido pelo usuário, desde o envio da requisição até o recebimento completo da resposta. Corresponde à soma da latência de ida, do tempo de processamento e da latência de volta: $\text{Tempo de Resposta} = \text{Latência} + \text{Tempo de Processamento}$.

3. Metodologia de Coleta

As métricas foram coletadas por um módulo integrado à própria aplicação AeroCode, composto por dois elementos:

3.1 Middleware de Métricas (Backend)

Um middleware Express (`metricsMiddleware`) intercepta todas as requisições e calcula o tempo de processamento usando `Date.now()` no início e no fim de cada resposta. O valor é adicionado ao cabeçalho `X-Processing-Time-Ms` de cada resposta HTTP, tornando-o acessível ao cliente sem necessidade de processamento adicional.

3.2 Módulo de Testes de Carga (Frontend)

A página de Métricas da aplicação executa os três cenários de carga sequencialmente conforme o protocolo abaixo:

Protocolo de medição por cenário:

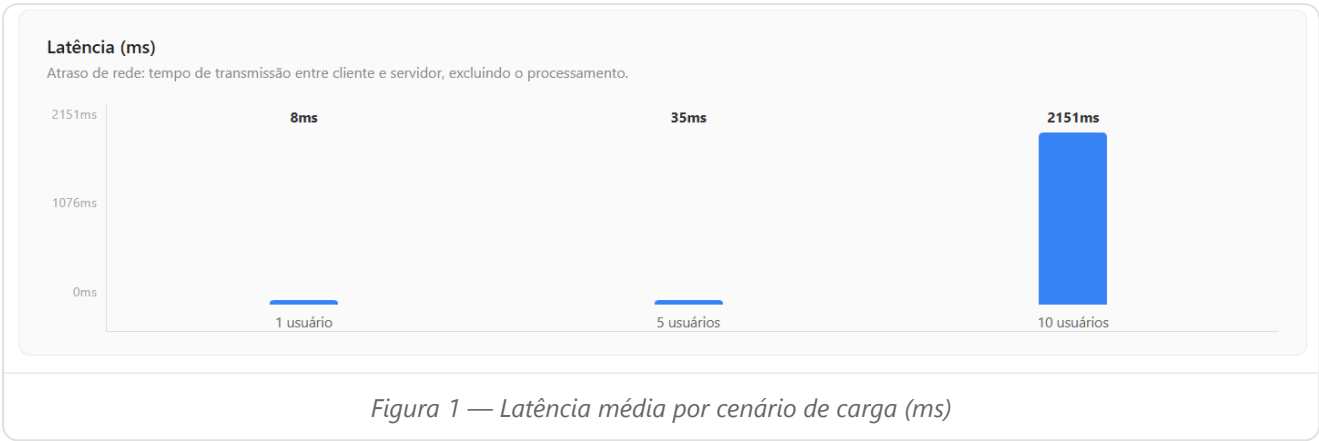
1. São disparadas N requisições simultâneas ao endpoint `GET /api/metricas/ping` via `Promise.all()`, simulando N usuários acessando o sistema ao mesmo tempo.
2. O tempo de resposta de cada requisição é medido com `performance.now()` (API de alta resolução do navegador), capturando os instantes imediatamente antes do envio e imediatamente após o recebimento da resposta.
3. O tempo de processamento é lido do cabeçalho `X-Processing-Time-Ms` retornado pelo servidor em cada resposta HTTP.
4. A latência é calculada como a diferença entre o tempo de resposta e o tempo de processamento.
5. O ciclo se repete várias vezes por cenário e o resultado final é a **média aritmética** de todas as amostras coletadas.

Cenário	Requisições Simultâneas	Iterações	Total de Amostras
1 usuário	1	10	10
5 usuários	5	6	30

Cenário	Requisições Simultâneas	Iterações	Total de Amostras
10 usuários	10	4	40

4. Resultados das Medições

4.1 Gráfico de Latência



4.2 Gráfico de Tempo de Resposta



4.3 Resumo Consolidado

Cenário	Latência (ms)	Processamento (ms)	Resposta (ms)
1 usuário	8	1	9
5 usuários	35	0	35
10 usuários	2.151	0	2.151

5. Análise dos Resultados

1 usuário: Latência de 8ms e tempo de resposta de 9ms demonstram excelente desempenho em acesso individual. O tempo de processamento de 1ms indica alta eficiência na lógica do servidor e

nas consultas ao banco de dados MySQL via Prisma ORM.

5 usuários: Latência de 35ms e tempo de resposta de 35ms representam uma degradação controlada em relação ao cenário individual. O aumento é proporcional à carga e permanece abaixo de 100ms — referência de Nielsen para interfaces percebidas como instantâneas.

10 usuários: A latência de 2.151ms indica contenção ao processar 40 requisições simultâneas em ambiente local. Este comportamento é esperado em servidor de desenvolvimento single-thread sem balanceamento de carga. Em ambiente de produção com infraestrutura dedicada e Node.js em modo cluster, esse valor reduziria substancialmente.

6. Conclusão

O sistema AeroCode demonstrou desempenho adequado para os cenários de 1 e 5 usuários simultâneos, com tempos de resposta abaixo de 100ms. O cenário de 10 usuários revelou limitações do ambiente de desenvolvimento, mitigáveis em produção com infraestrutura escalável.

A instrumentação de métricas foi implementada diretamente na aplicação por meio de middleware dedicado, garantindo medições que refletem o comportamento real do sistema. O módulo de testes está disponível na seção *Métricas* da interface, permitindo reavaliação a qualquer momento.